

UML: Diagramas de Classes

Introdução e Propósito

Os Diagramas de Classes são o **tipo de diagrama UML mais usado**, contendo conceitos de modelação base e avançados. Estes diagramas descrevem as "coisas" que o problema envolve e correspondem à análise entidade-relação clássica.

Análise Conceptual

A análise conceptual, onde os diagramas de classes se inserem, lida com *use cases*, **classes e relações** (estrutura de dados), interações (modelação de processos) e *constraints* (regras de negócio). Esta fase **não envolve** programação, implementação física ou o modelo relacional.

Componentes Principais

Um Diagrama de Classes é composto por: Classes, Propriedades, Atributos, Associações, Multiplicidade, Operações, Generalizações, Notas, Dependências e Restrições.

Classes e Propriedades

Classes

Classes são as "coisas" do problema. Para as descobrir, devem-se procurar os **substantivos do discurso** (ex: livros, leitores), que refletem entidades do mundo real e capturam o vocabulário do sistema. As classes devem ter "persistência" e são os blocos base do sistema.

- **Representação:** Uma caixa com 1, 2 ou 3 secções: nome/identidade, atributos e operações/métodos.

Atributos e Sintaxe

Propriedades são os "campos/atributos" de uma classe.

- **Sintaxe da Propriedade:** visibility name: type multiplicity = default {property-string}.
- **Visibility:** Indica se o atributo é **público (+)** ou **privado (-)**.
- **name:** Nome do atributo.
- **type:** Tipo de objeto ou dado que pode ser guardado no atributo.
- **default:** Valor predefinido na criação do objeto.

Atributos vs. Associações

Na teoria, atributos e associações são equivalentes.

- **Regra Geral:** Usar **atributos** para **tipos de dados simples** (datas, booleanos, inteiros).
- **Regra Geral:** Usar **associações** para **tipos de dados complexos** ou classes importantes do problema (ex: cliente, encomenda).

Associações e Multiplicidade

Associações

Associações representam **relações/ligações** entre classes. Podem ser encontradas através de **verbos do discurso** (ex: "leitor reserva revista") ou quando é necessário "navegar" de uma classe para outra.

- Podem ser entre diferentes classes (ex: professor leciona cadeira) ou **recursivas** (sobre a própria classe).
- **Associações Bidirecionais:** Podem ser representadas com um *label* (verbo ou frase) para permitir usar a associação em textos.

Multiplicidade

A multiplicidade indica **quantos objetos** podem pertencer a uma propriedade ou associação.

- **Comuns:** **1** (exatamente um), **0..1** (zero ou um, opcional) e ***** (zero ou mais/ilimitado). O valor ***** é um caso especial que significa **0..***.
- **Limite:** O limite inferior (≥ 0) e superior (> 0 ou *****) são especificados.
- **Atributos *Multivalued*:** Têm um limite superior > 1 , tipicamente *****, e representam um conjunto. A ordenação pode ser especificada (**ordered**) e, por defeito, os duplicados não são permitidos (**unique**).
- **Multiplicidade Default:** Se omitida, assume-se 1.

Operações e Generalizações

Operações

Operações são as ações ou métodos feitas por uma classe.

- **Sintaxe:** visibility name (parameter-list) : return type {property-string}.
- **Parâmetros:** Usam a notação direction name: type = default value. A direção pode ser **input** (**in**), **output** (**out**) ou **inout**.
- **Tipos de Operação:**
 - *Query*: Obtêm um valor, **sem** alterar o estado da classe.
 - *Modifier*: Operações que **mudam** o estado da classe.
 - *Setter/Getter*: Atribuem ou leem um valor de um atributo.

Generalizações (Herança)

É a relação entre classes com muitas semelhanças e algumas diferenças (ex: Cliente $\rightarrow$ Cliente Pessoal, Cliente Empresarial).

- A classe "filho" (subclasse/subtipo) **herda** todas as propriedades da classe "pai" (superclasse).
- **Princípio da Substituibilidade:** A classe "filho" deve poder substituir a classe "pai" (ex: um Cliente Pessoal é um Cliente).
- **Tipos:** Pode ser **simples** ou **múltipla** (herança de várias classes).
- **Descoberta:** Classes com atributos ou operações comuns, ou substantivos diferentes usados indistintamente em certos processos.

Conceitos Avançados e Estruturas Especiais

Classes Abstratas e Interfaces

- **Classe Abstrata:** Uma classe que **não pode ser instanciada**. Pode conter operações abstratas (sem implementação). É representada com o nome em *itálico* ou com a *keyword* {abstract}.
- **Interface:** Uma classe sem implementação, onde **todos** os elementos são abstratos. Mapeia diretamente para interfaces de linguagens de programação (Java, C#). Representada pela *keyword* <<interface>>.
- **Representação Alternativa de Interfaces:** A classe que **implementa** um interface é representada por um círculo (bola); a classe que **requer** um interface é representada por uma "ficha" ou "tomada".

Associações Avançadas e Classes Associativas

- **Agregações e Composições:** Tipos de associações que representam a relação "parte de".
- **Composição:** Forma mais forte. A classe "parte" **só pode ter um "dono"** e **não existe** se a classe "dona" for apagada.
- **Classes Associativas:** Permitem adicionar atributos, operações e outras características a uma relação ou associação (ex: atributos que descrevem a participação de uma Pessoa numa Reunião).

Restrições e Dependências

- **Restrições (*Constraint Rules*):** Especificam restrições globais sobre o sistema que os diagramas não conseguem cobrir sozinhas (ex: regras de negócio). São representadas como uma nota colocada entre {}.
- **Dependências:** Mostram que uma alteração na definição de um elemento ("fornecedor") implica alterações noutro elemento ("cliente"). Devem ser **minimizadas**, e deve-se ter cuidado com dependências cíclicas. São implícitas em associações navegáveis e generalizações.

Outros Elementos

- **Operações e Atributos Estáticos:** Referem-se à classe e não a objetos específicos. São representados com um **sublinhado**.
- **Keywords:** Usadas para distinguir diferentes tipos de utilização para a mesma entidade gráfica, colocadas entre << >> (aspas angulares) ou {}. Exemplos: <<interface>>, {abstract}.
- **Enumerações:** Conjunto de valores sem propriedades, usados para especificar os valores permitidos para um atributo. Representados com a *keyword* <<enumeration>>.
- **Visibilidade:** Inclui Público (+), Privado (-), **Protegido (#)** (visível à classe e subclasses) e **Package (~)** (visível dentro do mesmo pacote).

UML: Diagramas de Interação

Conceitos Fundamentais dos Diagramas de Interação

Os Diagramas de Interação modelam os **aspectos dinâmicos** do sistema. Eles descrevem a **colaboração entre vários objetos** e as **mensagens trocadas** num cenário de utilização específico.

Tipos de Diagramas de Interação

- **Diagramas de Sequência:** Focam-se primariamente no **tempo** e na **ordenação temporal das mensagens**.
- **Diagramas de Comunicação (ou Colaboração):** Focam-se na **organização dos objetos** e nos **links de dados** existentes entre eles.

Diagramas de Sequência

Participantes e Linhas de Vida

- **Participantes:** Representam instâncias ou objetos de classe. Podem ser nomeados sem referir a classe (ex: `anOrder`) ou referindo a classe (ex: `joao : Customer`).
- **Linha de Vida (*Lifeline*):** Uma linha vertical que representa o "tempo de vida" do participante no cenário modelado.
- **Caixa ou Barra de Ativação:** Uma barra vertical na linha de vida que indica o período de tempo em que o participante está ativamente a realizar uma operação.

Mensagens e Sincronização

- **Ordenação:** As mensagens são ordenadas de cima para baixo.
- **Mensagem Inicial (*Found Message*):** Não parte de um participante, tendo uma origem desconhecida, e é o ponto de partida de toda a sequência.
- **Mensagens Síncronas:** O chamador **espera** que a operação termine. A notação é uma **seta fechada** ().
- **Mensagens Assíncronas:** O chamador **prossegue imediatamente** após enviar a mensagem, sem esperar por uma resposta. São tipicamente usadas em sistemas *multi-thread*. A notação é uma **seta aberta** ($\rightarrow$).
- **Self-Message:** Mensagens que um participante envia a si próprio.

Estilos de Interação (Controlo)

- **Controlo Centralizado:** Todo o processamento está focado num único participante. Este estilo é **mais simples** de visualizar num diagrama.
- **Controlo Distribuído:** O processamento está espalhado por todos os participantes. Este estilo é **preferível** por ser mais orientado a objetos, promover maior modularidade, e juntar dados e comportamentos, aumentando as hipóteses de usar polimorfismo.

Criação e Eliminação de Objetos

- **Criação:** Uma mensagem (opcionalmente nomeada como "new") é enviada diretamente para a caixa do novo participante.
- **Eliminação:** Indicada com um **"X grande"** no fim da linha de vida. Se uma mensagem for enviada para esse 'X', um participante apaga outro; se o 'X' estiver no fim da própria linha de vida, o participante apagou-se a si próprio.
- **Uso:** Embora desnecessário em ambientes com *garbage collection*, é usado para fechar ou terminar operações, indicando que o objeto já não é preciso.

Lógica de Controlo (Ciclos e Condições)

Os Diagramas de Sequência **não são os mais indicados** para modelar estruturas de controlo complexas, mas utilizam *interaction frames*.

- **Interaction Frames (Caixas de Interação):** Marcam uma parte do diagrama, dividindo-o em fragmentos. Cada *frame* tem um operador e uma guarda (condição).
- **Operadores Comuns:**
 - **loop:** Indica um ciclo. A guarda controla a iteração.
 - **alt:** Indica múltiplos *frames* alternativos. Apenas os *frames* cuja guarda booleana seja verdadeira são executados (para modelar decisões condicionais).
 - **opt:** Indica um *frame* opcional, executado apenas se a condição associada for verdadeira.
 - **par:** Indica *frames* executados em paralelo.
 - **region:** Indica uma região crítica, permitindo apenas a execução de uma *thread* de cada vez.
- **Passagem de Dados:** Não tem notação *standard*, mas é feita através de parâmetros nas mensagens ou setas de retorno. Pode-se usar *data tadpoles* (setas com uma bola) para indicar o movimento dos dados.

Diagramas de Comunicação (Colaboração)

Foco e Elementos

- **Foco:** Concentra-se nos *links de dados* entre os participantes, que são colocados de forma livre no diagrama.
- **Links:** Representam as ligações entre participantes. Podem ser instâncias de associações de classes, ou *links* transientes (que só existem no contexto daquela interação) ou links locais («local»).

Numeração e Controle

- **Sequência:** As mensagens são **numeradas** para indicar a ordem correta de execução.
- **Numeração *Nested Decimal*:** O esquema 1, 1.1, 1.2, 1.3, 1.4, 1.5, 1.5.1 é o mais usado, pois permite saber qual é o âmbito da invocação das mensagens.
- **Lógica de Controle:** Não têm uma notação precisa para lógica de controle (como os *frames* de sequência). Usam **guardas** (expressões entre []) para indicar condições e **marcadores de iteração** (* adicionado à mensagem) para ciclos.

Comparação (Sequência vs. Comunicação)

- Ambos os diagramas são **equivalentes entre si** (pode-se derivar um do outro).
- **Diagrama de Sequência:** Usado quando o principal interesse é focar a **ordem e a sequência temporal** das chamadas.
- **Diagrama de Comunicação:** Usado quando se quer focar a **ligação e a organização** entre os participantes, sendo mais fácil para explorar diferentes alternativas.

UML: Diagramas de Atividades

Propósito e Contexto

Os Diagramas de Atividades são usados para modelar o **comportamento dinâmico** do sistema. O foco principal é nas **atividades**, distinguindo-se dos Diagramas de Interação (que focam objetos e mensagens) e dos Diagramas de Gantt (que focam o tempo e objetos).

Função dos Diagramas de Atividades

- Modelam/descrevem a **lógica procedimental, processos de negócio e fluxos de trabalho** de um sistema.
- São semelhantes aos fluxogramas.
- Podem ser vistos como um caso particular de diagrama de estados, onde os estados correspondem a atividades, ações ou operações.

Elementos Fundamentais

Os diagramas representam atividades e as **transições** entre elas (indicando quando e porquê ocorrem).

Início e Fim

- **Nó Inicial:** Ponto de partida do diagrama.
- **Nó Final:** Ponto final do diagrama.

Ações e Decomposição

- **Ações:** Uma ação concreta realizada por um objeto.
- **Decomposição de Ações:** Uma ação pode ser decomposta em **sub-atividades**. A ligação entre a ação "geral" e a sua decomposição é marcada pelo **símbolo *rake* (ancinho)** na ação do diagrama de nível superior.

Controlo de Fluxo

Estes elementos gerem a direção do fluxo de execução do sistema:

Concorrência (Paralelismo)

- **Fork/Bifurcação:** Representa a divisão de um único fluxo de entrada em **vários fluxos de saída concorrentes**, permitindo que as ações subseqüentes decorram em paralelo.
- **Join:** Marca o fim de um *fork*. Possui vários fluxos de entrada e um único fluxo de saída. O fluxo só **avança** depois de todas as entradas terem terminado.

Comportamento Condicional (Decisões)

- **Decisões:** Permitem comportamentos condicionais. Possui um fluxo de entrada e **vários fluxos de saída alternativos**.
- **Guardas:** Cada fluxo de saída alternativo deve ter uma **guarda** (condição booleana colocada entre []), e apenas o fluxo cuja guarda for verdadeira é seguido. Pode ser usada a guarda [else].
- **Merge:** Marca o fim de uma decisão. Possui vários fluxos de entrada alternativos e um único fluxo de saída que continua o processamento.

Organização e Eventos

Partições (*Swim Lanes*)

- **Propósito:** Os Diagramas de Atividades mostram o "que acontece", mas as **Partições** mostram **quem faz o quê**, especificando **qual classe** realiza a atividade.
- **Representação:** Dividem o diagrama em colunas ou faixas (*swim lanes*).

Sinais

- **Eventos Externos:** As atividades podem responder a **sinais**, que são eventos externos ao processo.
- **Escuta:** Uma atividade pode estar constantemente "à escuta" de um sinal.
- **Tipos:** Os sinais podem ser **temporais** ou indicar o envio/receção de uma informação.
- **Ponto de Entrada:** O ponto de entrada bem definido do diagrama (invocação de um método/rotina) provoca a transição de atividade.

Desenvolvimento Ágil de Software

Introdução e Filosofia Ágil

O desenvolvimento ágil de software surgiu no final dos anos 90 como resposta à necessidade de **desenvolvimento e entrega rápida** de software, dada a dinâmica acelerada dos negócios e a dificuldade em manter um conjunto de requisitos estável. O foco principal é reduzir drasticamente o tempo de entrega e a insatisfação com as metodologias "pesadas" (baseadas em planos) das décadas anteriores.

Características Chave

- A **especificação, desenho e implementação** estão **intercalados** e não em etapas separadas (como nos métodos baseados em planos).
- Os sistemas são desenvolvidos como uma **série de versões/releases** pequenas e frequentes.
- O foco é no **código funcional** em detrimento de documentação abrangente (documentação mínima).

Valores do Manifesto Ágil

Os métodos ágeis valorizam os seguintes aspetos (os itens à esquerda são mais valorizados do que os à direita):

1. **Indivíduos e interações** sobre processos e ferramentas.
2. **Software funcional** sobre documentação abrangente.
3. **Colaboração com o cliente** sobre negociação de contratos.
4. **Responder à mudança** sobre seguir um plano.

Princípios do Desenvolvimento Ágil

- **Envolvimento do Cliente:** O cliente deve estar intimamente envolvido, fornecendo e priorizando requisitos e avaliando as iterações do sistema.
- **Entrega Incremental:** O software é desenvolvido em incrementos, com o cliente a especificar os requisitos de cada um.
- **Pessoas e Não Processos:** As habilidades da equipa devem ser exploradas, permitindo que os membros desenvolvam os seus próprios métodos de trabalho.
- **Aceitar a Mudança:** Os requisitos devem ser esperados como mutáveis, e o sistema deve ser desenhado para acomodar essas alterações.
- **Manter a Simplicidade:** Foco na simplicidade do software e do processo, eliminando ativamente a complexidade.

Extreme Programming (XP)

O XP é um método ágil influente, criado no final dos anos 90, que leva a abordagem iterativa ao "extremo".

Ciclo de Releases XP

- Entregas de incrementos aos clientes a cada **2 semanas**.
- Podem ser criadas **várias versões por dia**.
- Uma nova versão só é aceite se **todos os testes passarem com sucesso**.

Práticas Chave do XP

- **Planeamento Incremental:** Requisitos registados em *story cards* (histórias de utilizador) e divididos em *tasks* (tarefas) pela equipa.
- **Small Releases:** A funcionalidade mínima útil é desenvolvida primeiro, adicionando-se incrementos nas versões subsequentes.
- **Simple Design:** É realizado apenas o desenho necessário para os requisitos atuais, sem antecipar alterações.

- **Test-First Development (TDD):** Escrever testes unitários automáticos **antes** de implementar a funcionalidade. Isto ajuda a clarificar os requisitos.
- **Refactoring:** Melhorias contínuas no código, feitas assim que são encontradas, mesmo sem necessidade imediata. O objetivo é manter o código simples e de fácil manutenção, tornando as alterações futuras mais fáceis de implementar. O XP defende que não se deve antecipar alterações, mas sim melhorar constantemente a estrutura.
- **Programação por Pares (*Pair Programming*):** Programadores trabalham juntos na mesma estação de trabalho. Isso promove a revisão informal do código, o refactoring e a partilha de conhecimento (reduzindo o risco de perda de membros da equipa).
- **Collective Ownership:** Todos os desenvolvedores trabalham em todas as áreas do sistema, e qualquer pessoa pode alterar qualquer parte do código.
- **Integração Contínua (*Continuous Integration*):** O trabalho concluído é imediatamente integrado no sistema, e todos os testes unitários devem passar após essa integração.
- **On-site Customer:** Um representante do utilizador final é membro a tempo inteiro da equipa, responsável por tomar decisões sobre os requisitos e por desenvolver os testes de aceitação para os cenários.

Scrum

Scrum é um método ágil com foco na **gestão** do desenvolvimento iterativo.

Fases do Scrum

1. **Fase Inicial:** Planeamento geral (objetivos, arquitetura).
2. **Sprint Cycles:** Desenvolvimento iterativo do sistema.
3. **Fase Final:** Documentação necessária e avaliação do projeto (*lessons learned*).

Terminologia e Estrutura

- **Sprint:** Uma iteração de desenvolvimento de **duração fixa**, geralmente **2 a 4 semanas**. O resultado é um incremento de produto potencialmente expedível.
- **Product Backlog:** Uma lista priorizada de todo o trabalho por fazer (funcionalidades, requisitos, tarefas de apoio).
- **Product Owner:** Responsável por identificar e priorizar os requisitos no *product backlog*, assegurando que o projeto satisfaz as necessidades críticas do negócio.
- **Development Team:** Grupo auto-organizado de desenvolvedores (≤ 7 pessoas) responsável por criar o software.
- **ScrumMaster:** Responsável por garantir que o processo Scrum é seguido, guiar a equipa e **protegê-la de interferências externas** durante o *sprint*.
- **Scrum:** Reunião diária curta (curta e presencial) da equipa para rever o progresso e priorizar o trabalho do dia.
- **Velocity:** Estimativa do esforço do *product backlog* que uma equipa consegue completar num único *sprint*. Ajuda a medir e estimar a capacidade.
- **Isolamento da Equipa:** Depois de escolhidas as funcionalidades do *sprint* (*Sprint selection phase*), a equipa de desenvolvimento é isolada do cliente (as comunicações passam pelo ScrumMaster).

Aplicação e Escalabilidade

Adequabilidade

Os métodos ágeis são mais adequados para:

- Projetos de **pequena e média escala** e sistemas à medida.
- Cenários onde o cliente pode ter uma **participação ativa** e a equipa é pequena e integrada, permitindo comunicação informal.

Manutenção e Escalabilidade

- **Escalar:** É difícil aplicar métodos ágeis a projetos muito grandes, de longa duração ou com equipas geograficamente distribuídas, onde a comunicação informal se torna desafiadora. É crucial manter os conceitos fundamentais, como versões frequentes e integração contínua.
- **Manutenção:** Os métodos ágeis não são altamente recomendados para a manutenção de software existente devido à **minimização da documentação** e à dificuldade de manter a continuidade da equipa de desenvolvimento original. A falta de documentação é um problema principal.

Agile vs. Plan-Driven

O equilíbrio entre métodos ágeis e baseados em planos depende de fatores como:

- A necessidade de **especificação muito detalhada prévia** (favorece Plan-Driven).
- A **viabilidade de entrega incremental** com *feedback* rápido do cliente (favorece Ágil).
- Sistemas com **tempo de vida longo** normalmente exigem documentação mais exaustiva (favorece Plan-Driven).
- Os métodos ágeis podem requerer uma **equipa de desenvolvimento de qualidade superior**.

Gestão de Configurações

Conceitos Fundamentais e Necessidade

A Gestão de Configurações (GC) refere-se às políticas, ferramentas e processos utilizados para **gerir alterações e mudanças** em sistemas de software que estão em constante evolução.

Importância da GC

- O software é visto como uma série de versões que implementam pedidos de alterações, correções de falhas e adaptações diversas.
- A GC é **essencial**, especialmente em **processos ágeis**, onde os sistemas são alterados e novas versões são criadas **várias vezes por dia**.
- É crucial para evitar perder o rasto das alterações e saber quais as mudanças que devem compor uma versão final do sistema.

Atividades Principais da GC

1. **Gestão de Alterações:** Rastrear pedidos de alteração, avaliar os custos de impacto dessas alterações e decidir quais devem ser implementadas.
2. **Gestão de Versões:** Manter o rasto das diferentes versões dos componentes do sistema, assegurando que as alterações de programadores distintos não interferem entre si. É o processo usado para gerir *codelines* e *baselines*.
3. **Criação do Sistema (*System Building*):** Processo de compilar e ligar componentes, dados e bibliotecas para construir um sistema executável.

4. **Gestão de Releases:** Preparar o software para ser entregue e manter o registo de todas as versões entregues ao cliente.

Terminologia e Estrutura de Versões

Itens de Configuração e Versões

- **Item de Configuração de Software (SCI):** Qualquer elemento do projeto (código, teste, documento) colocado sob controlo de configuração e com um identificador único.
- **Versão:** Uma instância de um item de configuração que difere de outras instâncias, possuindo sempre um identificador único.

Codelines e Baselines

- **Codeline:** Uma **sequência de versões** de um componente de software. As versões mais recentes são derivadas das mais antigas.
- **Baseline:** Uma **coleção controlada** de versões de componentes que, em conjunto, formam um sistema específico.
- **Controlo de Baselines:** As versões dos componentes que compõem uma *baseline* não podem ser alteradas, garantindo que é **sempre possível reconstruir uma versão específica** do sistema.
- **Mainline:** Uma sequência de *baselines* que representam as diferentes versões de um sistema.

Fluxos de Desenvolvimento

- **Branching (Bifurcação):** Criação de uma nova *codeline* a partir de uma versão existente. A nova *codeline* e a existente podem desenvolver-se de forma independente.
- **Merging (Fusão):** Criação de uma nova versão de um componente ao combinar versões separadas que se desenvolveram em diferentes *codelines*. Se as alterações envolverem diferentes partes do código, a fusão pode ser automática.
- **Workspace:** Área de trabalho privada de um programador, onde o software pode ser modificado sem afetar outros desenvolvedores.

Sistemas de Gestão de Versões (VCS)

Os VCS identificam, armazenam, controlam acessos e gerem o histórico das diferentes versões do sistema.

VCS Centralizados (Ex: SVN)

- Utilizam um **repositório central** que gere todas as versões de todos os componentes.
- **Fluxo:** Os programadores fazem *check-out* (cópia) dos componentes do repositório para a sua área de trabalho e *check-in* (envio) das alterações de volta para o repositório.
- Se vários programadores copiarem o mesmo componente, o VCS alerta para a possibilidade de conflito.

VCS Distribuídos (Ex: Git)

- Utilizam um **repositório distribuído**, sendo o *master* criado num servidor.
- **Fluxo:** Em vez de fazer *check-out* de ficheiros específicos, o programador cria um *clone* do repositório, que se torna o seu **repositório privado**.
- As alterações são feitas no clone local e guardadas com *commit* no repositório privado. Depois, as alterações são enviadas para o repositório do projeto (*push*).

- **Vantagens:** Permite um **mecanismo de *backup*** (se o repositório mestre falhar), permite trabalhar em modo ***off-line*** e permite **compilar e testar todo o sistema localmente**.
- **Desenvolvimento Open Source:** É essencial, pois permite que várias pessoas trabalhem sem coordenação central, enviando alterações para um repositório público até que o gestor do sistema decida integrá-las no repositório *master*.

Gestão de Armazenamento

- **Sistemas Antigos:** Usavam ***deltas*** (lista de diferenças entre versões) para minimizar o espaço em disco, que era caro. A reconstrução de uma versão exigia aplicar a cadeia de *deltas* correspondente.
- **Sistemas Modernos (Git):** Priorizam a velocidade. Não usam *deltas*, mas sim **algoritmos de compressão** (ficheiros e metadados) e evitam armazenar ficheiros duplicados. Utilizam ***packfiles*** (vários ficheiros pequenos combinados num único ficheiro indexado) para otimizar o acesso.

Criação do Sistema e Integração Contínua

Plataformas de Construção

- **Plataforma de Desenvolvimento:** Onde os programadores trabalham (usando IDEs e compiladores). O programador faz *check-out* do código para o seu *workspace* privado e testa localmente antes de fazer *commit*.
- **Build Server:** Usado para criar as versões executáveis (*releases*) finais. O programador faz *check-in* do código no VCS, e o *build server* faz *check-out* para construir o sistema.
- **Target Environment:** Plataforma onde o sistema será executado pelo utilizador.

Integração Contínua (CI) em Métodos Ágeis

A CI é um processo automatizado crucial para garantir a qualidade em ambientes ágeis:

1. O programador faz *check-out* da *mainline* para o seu ambiente privado.
2. O sistema é construído e todos os testes são corridos automaticamente no ambiente privado para garantir que passam. Se falharem, o responsável pela última *baseline* é notificado.
3. O programador faz as alterações e garante que todos os testes voltam a passar localmente.
5. Após os testes passarem no ambiente local, o programador faz *check-in* no VCS.
6. O *build server* constrói o sistema e corre todos os testes.
7. Se os testes passarem no *build server*, as alterações são registadas como uma **nova *baseline* na *mainline*** do sistema.

Vantagens e Desvantagens da CI

- **Vantagem:** Permite a descoberta e correção de erros causados pela interação entre vários programadores o **mais cedo possível**. A versão mais recente na *mainline* é sempre a versão atual e funcional do sistema.
- **Desvantagem:** Se o sistema for muito grande, a construção e teste podem demorar muito tempo.

Introdução ao Git

Introdução e Configuração

O Git é um **Sistema de Gestão de Versões Distribuído** que permite a monitorização e controlo de alterações.

Configuração

- A configuração pode ser lida e especificada através da ferramenta `git config`.
- **Níveis de Configuração:** Sistema (`/etc/gitconfig`), Utilizador (no diretório `home` do utilizador) ou Repositório (no diretório `.git/config`).
- **Identidade Pessoal:** É fundamental especificar o nome e o email para identificar os *commits*:

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

- **Editor de Texto:** Pode ser definido globalmente (ex: `$ git config --global core.editor emacs`).
- **Verificação:** A configuração pode ser verificada com `$ git config --list` ou especificando a chave (`$ git config user.name`).

Obter um Repositório

Existem duas formas principais de obter um repositório Git:

1. Criar um Repositório numa Diretoria Existente:

- Inicializar o Git: `$ git init`.
- Começar a "monitorizar" (tracking) ficheiros usando `$ git add`.
- Fazer o primeiro *commit* (`$ git commit -m 'mensagem'`).

2. Clonar um Repositório Existente:

- Usa-se o comando `$ git clone [url]` (opcionalmente com um nome de diretoria).
- Suporta URLs HTTP, HTTPS, Git ou SSH.
- Quando um repositório é clonado, todos os ficheiros ficam imediatamente **monitorizados** (*tracked*) e **não modificados** (*unmodified*).

Registrar Alterações e Gestão de Ficheiros

Estado dos Ficheiros

Os ficheiros no Git podem estar em dois estados principais:

1. **Untracked (Não monitorizado):** Ficheiro novo que o Git ainda não está a acompanhar.
2. **Tracked (Monitorizado):** Ficheiro que o Git já conhece, que pode estar:
 - *Unmodified* (Não modificado).
 - *Modified* (Modificado): Alterado, mas não na área de *staging*.
 - *Staged* (Pronto para *commit*): As alterações foram marcadas para inclusão no próximo *commit*.

Comandos de Estado e Staging

- **Verificar Estado:** `$ git status` ou a versão resumida `$ git status -s`. A versão resumida utiliza marcadores como **A** (*staged for commit*), **M** (*modified*) ou **??** (*new/untracked files*).
- **Monitorizar/Stage:** `$ git add <file>`. Este comando "faz stage" do ficheiro **exatamente no estado em que se encontra** no momento da execução. Se o ficheiro for alterado após o `git add`, será necessário correr o `git add` novamente para incluir as novas alterações.
- **Ver Alterações:** `$ git diff` mostra as alterações **ainda não marcadas para stage**.

Commits

- **Realizar Commit:** `$ git commit` (abre o editor) ou `$ git commit -m "mensagem"`.
- **Evitar Staging Area:** Para fazer *commit* de todos os ficheiros *tracked* que foram modificados, sem passar pelo *staging area*: `$ git commit -a -m "mensagem"`.

Gestão e Ignorar Ficheiros

- **Apagar Ficheiros:** Para apagar um ficheiro e removê-lo dos *tracked files*: `$ git rm filename`.
- **Fazer Untrack:** Para remover um ficheiro do repositório, mas mantê-lo na diretoria de trabalho: `$ git rm --cached README`.
- **Mover Ficheiros:** `$ git mv file_from file_to`.
- **Ignorar Ficheiros:** O ficheiro `.gitignore` lista ficheiros ou diretórios que o Git deve ignorar, especificados através de padrões de nomes (*glob expressions*), tais como `*.txt` ou `build/`. O uso de `!` nega uma exclusão anterior.

Histórico e Desfazer Ações

Histórico

- **Consultar Histórico:** `$ git log` (mostra o *hash* do *commit*, autor, data e mensagem).

Desfazer Coisas

- **Corrigir o Último Commit:** Se ficheiros forem esquecidos ou a mensagem estiver incorreta, pode-se corrigir o último *commit*: `$ git commit --amend`.
- **Fazer Unstage de um Ficheiro:** Para remover um ficheiro da área de *staging*, mas manter as alterações na diretoria local: `$ git reset HEAD <file>`.
- **Descartar Alterações Locais:** Para reverter um ficheiro modificado para o estado do último *commit*: `$ git checkout -- filename`.

Remotes e Colaboração

Remotes e Operações

- **Remotes:** Versões do projeto alojadas noutra local, essenciais para a colaboração.
- **Listar Remotes:** `$ git remote` ou `$ git remote -v`.
- **Adicionar Remote:** `$ git remote add <shortname> <url>`.
- **Fetching:** `$ git fetch [remote-name]` descarrega todos os dados do projeto do remote, mas não junta (*merge*) os dados remotos com os branches locais.
- **Pulling:** `$ git pull` executa *fetch* seguido de *merge*.
- **Pushing:** `$ git push [remote-name] [branch-name]` envia as alterações locais para o remote.

Tags e Branches

Tags

- **Propósito:** Marcar pontos específicos na história do projeto, como **releases** ou versões.
- **Criação:** `$ git tag <tagname>` (para o *commit* atual). Pode-se também usar o *hash* de um *commit* anterior.

- **Partilha:** Por defeito, `git push` não transfere *tags*. É necessário enviar tags explicitamente: `$ git push origin --tags`.
- **Checkout:** Para criar um *branch* novo baseado numa *tag* específica: `$ git checkout -b branchname tagname`.

Branches

- **Definição:** Um *branch* é um **apontador móvel** para um *commit*.
- **Branch Default:** O *branch* por defeito é o **master**.
- **HEAD:** Apontador especial que indica qual é o *branch* atual em que estamos a trabalhar.
- **Criação e Mudança:** `$ git branch testing` cria um *branch*; `$ git checkout testing` muda para esse *branch*.
- **Atalho:** `$ git checkout -b nome_branch` cria e muda para o novo *branch*.

Branching e Merging (Fusão)

- **Fluxo de Trabalho:** É comum criar um *branch* de tópico (*topic branch*) para cada tarefa/problema (*issue*).
- **Merge *Fast-Forward*:** Ocorre quando o *branch* a ser fundido está diretamente à frente do *branch* principal, resultando apenas no avanço do ponteiro.
- **Apagar Branch:** Depois de fundido, um *branch* pode ser apagado com `$ git branch -d hotfix`.
- **Conflitos:** Ocorrem quando o *merge* não é automático (alterações no mesmo local do código).
- **Resolução de Conflitos:** O *merge* falhado não cria um *commit* automático. Os conflitos são assinalados no ficheiro com `<<<<<< HEAD` e `>>>>>> branchname`.
- **Conclusão do Merge:** Após a reparação manual do conflito, deve-se usar `git add` para marcar a resolução e `git commit` para concluir a fusão.

Fluxo de Branches de Longa Duração

- **Master:** Branch principal, geralmente para o código de **produção**.
- **Develop:** Branch de desenvolvimento, para integração e teste de estabilidade. Os *topic branches* são tipicamente fundidos neste *branch*.
- **Topic:** Branches criados para desenvolver funcionalidades ou corrigir problemas específicos, sendo de curta duração.